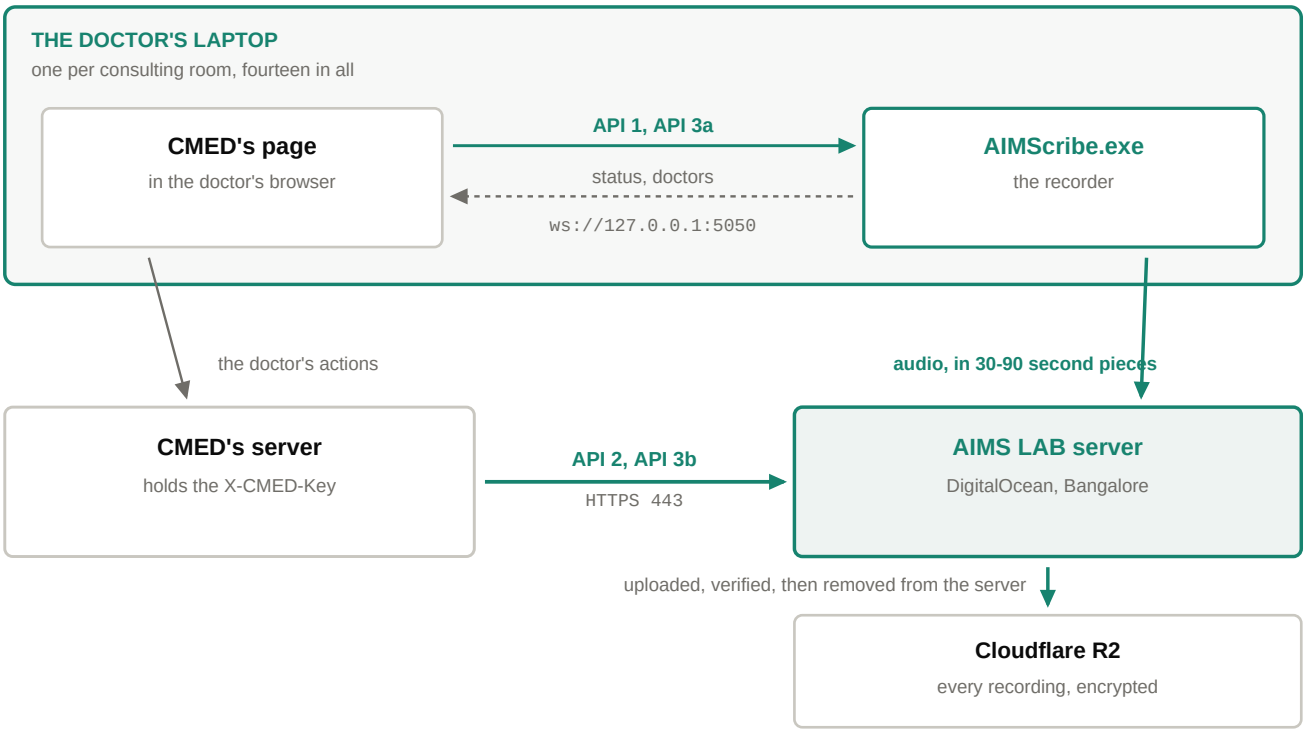
CMED → AIMS LAB — Software Integration

Document 1 of 2 — for CMED's software, database and data team · The APIs, the keys, the fields and the data model · AIMScribe v3 · AIMS LAB, United International University · 16 October 2026

	The two documents
1. This one	**Software integration** — the four APIs, the key, the exact endpoints, every field, every reply code, where the data lands, and how to test it. **For the team writing the code**
2. The other	**Server deployment** — the machine, the region, DNS, TLS, ports, the services, health checks and monitoring. **For the team helping us deploy**

What this integration is, in one paragraph

AIMScribe records the consultation. It runs as a small program on **the same laptop** as the doctor's browser. CMED's page tells that program when a patient is opened and when the prescription is built, so it knows where one consultation ends and the next begins. Separately, **CMED's server** sends us the patient's information and the prescription, so the recording has a clinical record attached. **Audio never goes to CMED, and nothing clinical is ever served back to CMED.**



Audio never reaches CMED. Nothing clinical is ever served back to CMED.
The two downward paths never meet: the recorder sends audio to us, and CMED's server sends us the clinical record. Neither party ever receives the other's data, and there is no endpoint by which it could.
CMED's page never learns the AIMS LAB address either - only CMED's own server holds it.

Every rule here is taken from the program that validates these messages on arrival. **If this document and the software disagree, the software is right and this is a bug** — please tell us.

Contents

Part	Covers	Sections
A	The keys and the exact endpoints — **start here**	1–3
B	The five fields, and how the four messages relate	4–5
C	The two browser signals — API 1 and API 3 Channel A	6–11

D	The two server messages — API 2 and API 3 Channel B, **and the outbox for outages**	12–17
E	**Previous prescriptions** — sending them, and asking what we already hold	18–19
F	The data model — where every field lands	20–21
G	Testing, handover, and one page to remember	22–25

PART A

The keys and the exact endpoints

Copy these. Everything after this explains them

1 · The gateway

```
BASE_URL = https://<aims-host>          <-- AIMS LAB sends this with the key

Transport    HTTPS only, TLS 1.2 or 1.3, port 443. HTTP redirects to HTTPS
Encoding     JSON, UTF-8
Body limit   1 MB
Auth         X-CMED-Key: <key>     on the two clinical endpoints only
```

Hold the host in configuration, not compiled in. You will get a staging host first and a production host later, and they differ.

2 · The authentication, in full

	Channel B — your server to ours	Channel A — your page to the recorder
Credential	`X-CMED-Key` **header**	**none — and none is possible**
Why	Server to server, so a secret is safe	A secret in a web page is readable by anyone who opens the page
What authorises you	the key	**your page's exact web address**, on our allow-list, plus loopback-only, plus same-machine
Issued by	AIMS LAB, once, out of band. **Shown once and not recoverable**	you send us the address; we register it
Scope	one key for CMED, covering all seven clinics	one entry per address
Rotation	ask us — new key issued before the old is revoked, so there is no window where both fail	n/a
Where it must live	**your server's secret store or environment**	n/a

The key must never reach a browser — not in page source, not in a bundled script, not in a mobile app. A wrong or missing key returns **401 INVALID_KEY**, which is a hard stop: do not retry it, alert your team and contact us.

The address allow-list is a blocking prerequisite, and it is exact
Send AIMS LAB **every web address your page will be served from**, production and staging and preview. An address not on the list is refused with `ORIGIN_NOT_ALLOWED` and the connection never opens, so **nothing works at all**. It must match character for character: `https://ehr.aaloclinic.com` and `https://www.ehr.aaloclinic.com` are different, so are `http` and `https`, and so is any non-default port.

3 · Every address CMED calls

#	Method	Full address	Auth	Body	OK
1	**GET**	`https://<aims-host>/health`	**none**	none	**200**
2	**GET**	`https://<aims-host>/`	none	none	200
3	**POST**	`https://<aims-host>/api/v2/clinical/patient-information`	**key**	JSON	**202**
4	**POST**	`https://<aims-host>/api/v2/clinical/prescription`	**key**	JSON	**202**
5	**GET**	`https://<aims-host>/api/v2/clinical/patient/{patient_id}/visits`	**key**	none	200

Protocol	Full address	Auth	Who
WebSocket	`ws://127.0.0.1:5050/ws`	address allow-list	your page, in the doctor's browser

Message on that socket	Which API	Sent when
`{"command": "start", "trigger": {…}}`	**API 1**	the doctor opens a patient
`{"command": "prescription_built", …}`	**API 3, Channel A**	the doctor presses Build Prescription
`{"command": "doctors"}`	—	**on page load** — returns this PC's clinic code
`{"command": "status"}`	—	optional; the recorder also pushes status unprompted

All seven, one line each, to copy

```
GET  https://<aims-host>/health
GET  https://<aims-host>/
POST https://<aims-host>/api/v2/clinical/patient-information
POST https://<aims-host>/api/v2/clinical/prescription
GET  https://<aims-host>/api/v2/clinical/patient/{patient_id}/visits
WS   ws://127.0.0.1:5050/ws  → "start"
WS   ws://127.0.0.1:5050/ws  → "prescription_built"
```

One GET, and it returns no clinical content. The visits list (section 19) gives identifiers and a fingerprint of each prescription, so CMED can ask what we hold before sending history — never a name, a medicine or a diagnosis. **There is no DELETE anywhere. Everything else under `/api/v2/` is closed to CMED's key**; those endpoints belong to the recorders and need each recorder's own device credential.

PART B

The five fields

The one rule that decides whether the whole integration works

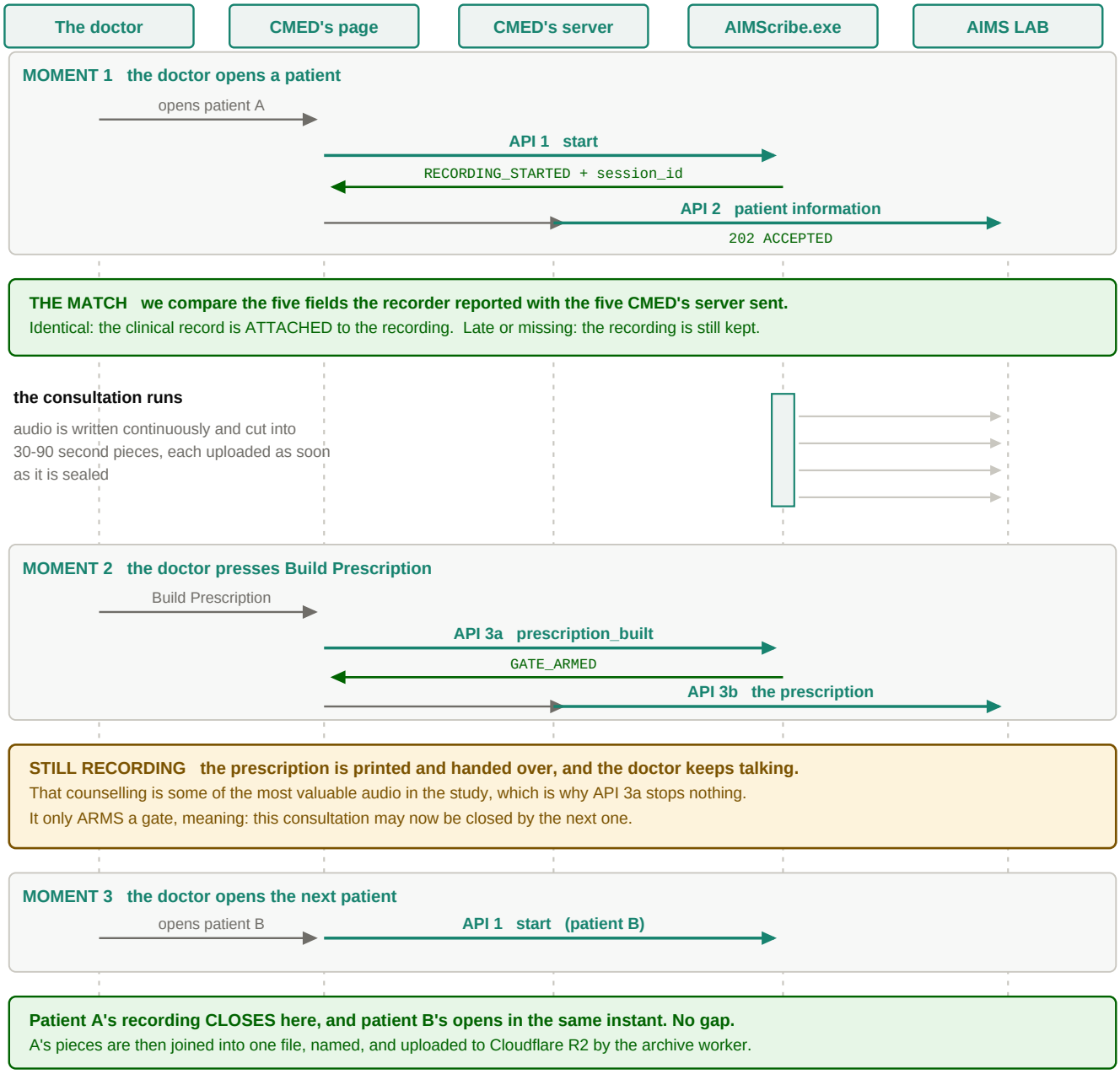
4 · Three APIs, four messages

	What	From	When	Part
API 1	Start recording	the browser	the doctor opens a patient	C
API 2	**Patient information**	**your server**	the same moment	D
API 3 Channel A	Prescription built	the browser	the doctor presses Build Prescription	C
API 3 Channel B	**The prescription**	**your server**	the same moment	D

They go in pairs. Two moments, four messages. **API 3 is one event told to two places** — the recorder, so it knows the consultation has reached its end, and our server, so we receive the prescription. The earlier integration guide calls these 6a and 6b; this document says Channel A and Channel B. Three APIs, four messages.

4.1 · One whole consultation, start to finish

Read this once and the rest of the document is detail. Time runs down the page; each column is one party.



Three things to take from it. The four messages happen at only **two moments**, in pairs. **The match** — the step CMED cannot see — is what attaches CMED's clinical record to the recording; the recording itself is kept whether or not it matches. And the recorder **keeps recording through the prescription**, closing only when the next patient is opened.

5 · The five fields

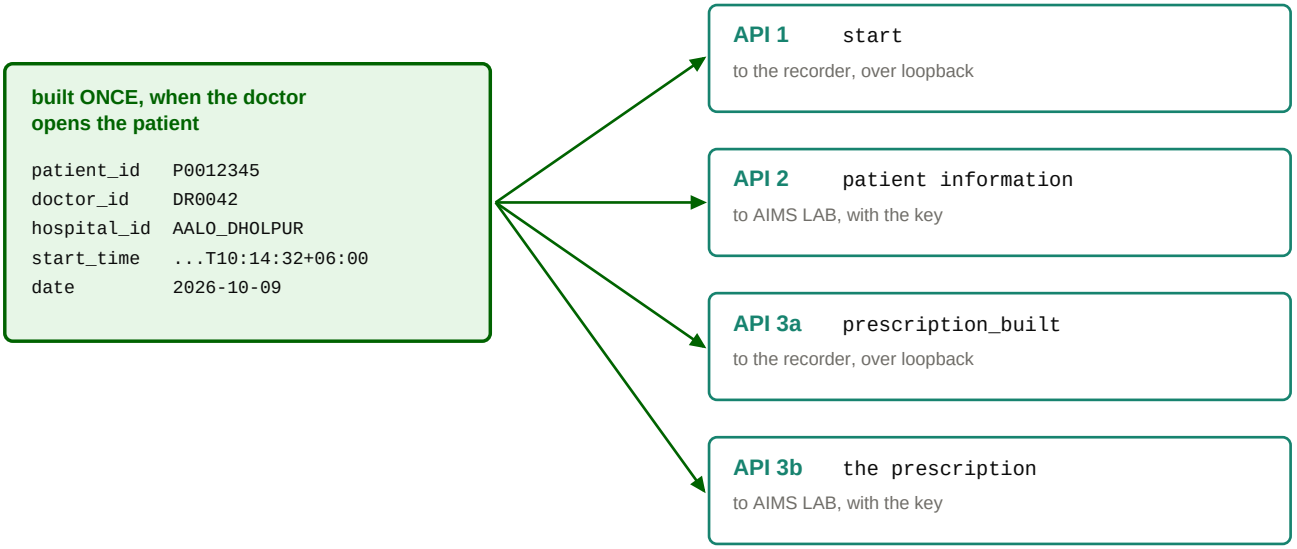
All four messages about one consultation carry these same five fields, with **exactly the same values, character for character**.

```
{
  "patient_id":  "P0012345",
  "doctor_id":   "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time":  "2026-10-09T10:14:32+06:00",
  "date":        "2026-10-09"
}
```

Field	Type	Rule
patient_id	text	Letters, digits, hyphen, underscore. 1–64 characters. **No spaces**
doctor_id	text	same rule
hospital_id	text	same rule. **Read it from the recorder — section 10**
start_time	text	Date and time **with the time-zone offset**, e.g. `+06:00`
date	text	`YYYY-MM-DD`, the clinic's local date

Three mistakes that break the match
1. Sending the prescription's own time as start_time. Both halves of API 3 repeat the **original** start_time from when the patient was opened. This is the most common integration mistake, and it is silent: both messages succeed and the prescription simply never finds its recording.
2. Leaving off the offset. `2026-10-09T10:14:32` is rejected.
3. A space or other character in an identifier. These become folder names on disk.

5.1 One object, four messages



Pass the same object to all four. Never rebuild it and never recompute the time.
In API 3a and API 3b, start_time is still the moment the PATIENT WAS OPENED - not the moment the prescription was built. That single mistake is the most common way this integration fails, and it is silent: every message succeeds, and the prescription simply never finds its recording.
Take hospital_id from the recorder's doctors command, not from your own settings.

Build the five fields once, into one object, when the doctor opens the patient. Keep it for the whole consultation and pass the same object to all four messages. Never rebuild them and never recompute the time. Do that and the match cannot fail; section 23 shows it in code.

PART C

The two browser signals

API 1 and API 3 Channel A - and the gate

6 · How the page connects

	Detail
Address	`ws://127.0.0.1:5050/ws` — the same on every clinic PC
Why `ws://` and not `wss://`	It never leaves the machine. Loopback needs no certificate and the browser permits it
Protocol	One WebSocket held open for the clinic session. JSON, UTF-8
Largest message	64 KB
When to connect	when the page loads; reconnect quietly if it drops
Firewall changes	**none**, on CMED's side or the clinic's

7 · Every message and reply

```
CMED sends:
{ "command": "start", "request_id": "abc-123", ... }

Exactly one reply per message:
{
  "event":      "ack",                <-- "ack" below 400, "error" from 400
  "command":    "start",
  "status":     200,
  "code":       "RECORDING_STARTED",   <-- ACT ON THIS
  "message":    "Recording started.",  <-- for a human; may be reworded
  "request_id": "abc-123",
  "data":       { ... }
}
```

Act on `code`, never on `message`. The code is a promise; the wording is not.

8 · API 1 — a patient has been opened

```
{
  "command": "start",
  "request_id": "open-P0012345-1",
  "trigger": {
    "patient_id": "P0012345", "doctor_id": "DR0042",
    "hospital_id": "AALO_DHOLPUR",
    "start_time": "2026-10-09T10:14:32+06:00", "date": "2026-10-09"
  }
}
```

The microphone starts **immediately**, before any permission check finishes, so nothing of the consultation is missed. What comes back:

```
"data": {
  "session_id": "01M3MK4J4XEDG5NDKE4H5B5XA2",   <-- KEEP THIS
  "started_at": "2026-10-09T10:14:32.865Z",
  "armed": false,
  "supersedes": "01M3MEYY...",       <-- the session this one closed, or null
  "authorisation": "granted",        <-- granted | pending | held
  "confirmation": "confirming"
}
```

9 · Every reply to API 1

Code	Meaning	What CMED should do
RECORDING_STARTED (200)	Recording, permission confirmed. With `data.continued: true`: a recording was already running and carries on	**Store the session_id.** Show it is recording. Send API 2 as usual
RECORDING_PROVISIONAL (202)	**Recording.** Permission still being checked	**Treat as success.** Normal, not a warning
SESSION_ALREADY_ACTIVE (409)	This patient is already being recorded, and the prescription was built	Nothing; you probably sent it twice, or reopened the profile
MISSING_FIELD (400)	A field is missing or malformed	Fix and resend
INVALID_IDENTIFIER (400)	An id has disallowed characters	Fix and resend
DEVICE_NOT_ENROLLED (423)	This PC is not registered with AIMS LAB	**Stop.** Contact AIMS LAB
ORIGIN_NOT_ALLOWED (403)	Your page's address is not on the allow-list	**Send us the address.** Section 2
AGENT_NOT_READY (503)	The recorder is starting up, or the microphone is busy	**Nothing is required** — the recorder keeps trying by itself (section 11A). Resending is harmless
GATE_NOT_ARMED, PATIENT_NOTED, CLINIC_MISMATCH, AUTHORISATION_FAILED	*Sent only by older recorders*	Treat as section 11A says; this version accepts the click and keeps recording

RECORDING_PROVISIONAL is the one people get wrong. The microphone is live and audio is being written; the server has simply not finished confirming permission, which normally takes a fraction of a second. Show the same "recording" state as for RECORDING_STARTED. Do not warn and do not retry.

A provisional start is never stopped by the server afterwards. If our server says no — a clinic not yet mapped, a configuration being fixed — the recording carries on and is **kept on the laptop**; `status` shows `authorisation: "held"`, AIMS LAB is alerted, and the recorder asks again every five minutes until the fix releases it. **Nothing is deleted and the page has nothing to do.** If the server cannot be reached at all, the reply comes at once rather than after a wait.

10 · Never hardcode the clinic code — ask for it

Every AIMScribe laptop is enrolled to exactly one clinic. A recording started with another clinic's code is **not uploaded**: it waits on the laptop, and AIMS LAB is alerted, until someone fixes the code. **So do not store clinic codes in your page — read them off the laptop on page load.**

```
send:  { "command": "doctors", "request_id": "who-is-this-pc" }

reply: "data": {
         "hospital_id": "AALO_DHOLPUR",     <-- use THIS as hospital_id
         "doctors": [
           { "doctor_id": "DR0042", "full_name": "Dr Rahman" },
           { "doctor_id": "DR0051", "full_name": "Dr Akter" }
         ]
       }
```

What comes back	Meaning	What the page should do
a code, and doctors listed	Normal	**Proceed.** Use the code; fill the doctor selector
a code, `doctors: []`	**Also normal** — a new clinic, or our server briefly unreachable	**Proceed.** Let the doctor be typed. **An empty list must never block a clinic**

`hospital_id: null`	**This PC is not enrolled**	**Do not send API 1.** Say so and contact us

This makes a clinic mismatch impossible rather than something to handle, and lets us rename a clinic without CMED changing anything. It already caused one failure in testing: a page was sending an old code after a clinic was renamed, and every recording was refused until the page was rebuilt. **Re-send `doctors` whenever the socket reopens.**

11 · API 3 Channel A, the gate, and status

```
{
  "command": "prescription_built",
  "request_id": "built-P0012345-1",
  "patient_id": "P0012345",
  "session_id": "01M3MK4J4XEDG5NDKE4H5B5XA2"    <-- from API 1
}
```

Field	Required	Rule
command	**yes**	`prescription_built`
patient_id	**yes**	**the patient the prescription was written for**
session_id	**yes**	**the one the recorder gave you** — in API 1's reply, or status
occurred_at	no	date and time with offset

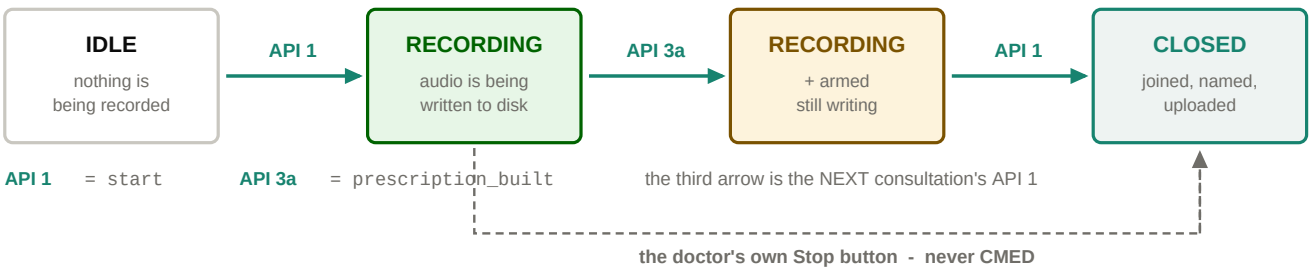
Code	Meaning	Do
GATE_ARMED (200)	Accepted, for any patient opened during this recording	nothing
GATE_ALREADY_ARMED (200)	Sent twice	nothing; harmless
PATIENT_MISMATCH (409)	A patient not opened during this recording, or another session	check the session_id
NO_ACTIVE_SESSION (409)	Nothing is recording	the consultation already ended

Building the prescription does NOT stop the recording

The doctor builds it, prints it, hands it over — and then **keeps talking**. That counselling is some of the most valuable audio in the study.

So `prescription_built` ends nothing. It **arms a gate**: this consultation has reached its end, so the next patient is allowed to close it. **The recording stops when the next patient is opened** — your next API 1, typically twenty to thirty seconds later — or by itself two minutes after the prescription if nobody else is opened (it waits up to ten seconds more for a pause, so no sentence is cut). That keeps the counselling and leaves the doctor's time between patients unrecorded.

11.1 What the recorder is doing, state by state



The third state is the one that gets missed.
prescription_built does not stop the recording. It moves the recorder from RECORDING to RECORDING + armed, which means "this consultation may now be closed by the next one". Audio keeps being written throughout.
An API 1 before that is accepted into the same recording; the server files each part under the right patient.

The doctor does	CMED sends	The recorder does
Opens patient A	API 1 + API 2	starts recording A
Consults	—	recording
Presses Build Prescription	**API 3, both channels**	**keeps recording.** Arms the gate
Prints, hands over, counsels	—	**still recording — the valuable part**
Opens patient B	**API 1 + API 2**	**closes A**, starts B

11.2 Another patient opened before the prescription — always accepted

The doctor does	CMED sends	The recorder answers, and does
Opens patient W by mistake	API 1 + API 2 (W)	RECORDING_STARTED — records
Notices, opens the right patient R	API 1 + API 2 (R)	**RECORDING_STARTED**, same session_id, `continued: true` — keeps recording, writes down R and the second
Builds R's prescription	**API 3, both channels, for R**	**GATE_ARMED** — keeps recording
Opens the next patient	API 1 + API 2	closes this recording, starts the next

Every click is accepted, and nothing new is asked of CMED. A patient opened before the prescription is built may be a glance at another file, a wrong profile corrected, or the next patient after a follow-up that needed no prescription — that cannot be known at the moment of the click. So the recorder gives the success reply your code already handles, keeps recording, and writes down who was opened and when. **Carry on exactly as for any patient**: send their API 2, and API 3 for whoever the prescription is for, with that patient's own five fields and the session_id the recorder gave you. When the recording ends, the AIMS LAB server reads what followed each click — a prescription, a return to the first patient, how long each was open — and files each part of the audio under the right patient. A wrong profile picked first loses nothing; a follow-up with no prescription gets its own file; a glance is filed with the patient the doctor went back to. Anything it cannot settle is kept and shown to AIMS LAB for review.

If the doctor finishes for the day or goes to lunch, they press Stop on the recorder's own on-screen control. **CMED sends nothing, and must not try to.** The socket also accepts `stop`, `pause` and `resume`, but **these are not CMED's to use**: each records a clinical reason that becomes part of the permanent record, including "the patient did not consent", which erases the recording everywhere. **Send only `start` and `prescription_built`.**

Status, pushed to you unprompted

```
{
  "event": "status",
  "state": "recording",           <-- idle | recording | paused | closing
  "is_recording": true, "is_paused": false,
  "session_id": "01M3MK4J...", "patient_ref": "P0012345",
  "armed": false,                 <-- has the prescription signal arrived?
  "authorisation": "granted",     <-- granted | pending | held
  "confirmation": "confirmed",
  "duration_seconds": 182.4,
  "level": 0.42,                  <-- 0.0 to 1.0, how loud the room is
  "connected_clients": 1
}
```

Show `level` as a small meter. A microphone that is unplugged, muted in Windows, or set to the wrong input records perfect silence and **nothing on screen looks wrong**. Without a meter that is discovered when somebody plays the file back, possibly weeks and hundreds of consultations later. It is the cheapest safeguard in the whole integration.

11A · When the recording does not start — recovering without the doctor

The doctor opens a patient and the page sends API 1 — but the recorder was not running yet, or the socket had dropped a second earlier, or the microphone was briefly busy, or the recorder crashed mid-consultation. **The doctor must never have to notice, and must never have to re-open the patient.** Two safety nets make that true: the recorder's own, and the page's.

Net 1 — the recorder recovers by itself

Microphone busy at the start: the page gets `AGENT_NOT_READY`, and the recorder **keeps the trigger and tries again every five seconds, for up to fifteen minutes**, until it records. It stops trying the moment CMED sends another patient or the doctor presses Stop. AIMS LAB is alerted either way.

Recorder crashed or restarted mid-consultation: it is back within a minute and, **if that consultation was live in the last ten minutes, records it again under the same five fields** by itself, with the gate as it was. A consultation from earlier, or from yesterday, is never resumed.

Server says no: the recording carries on and is kept on the laptop (section 9). Nothing for the page to do.

Net 2 — the page reconciles

The recorder can only recover a start it received. If the socket was down when the patient was opened, only the page knows. So the page does not fire and forget: it always knows what **should** be happening — which visit is open, and whether its prescription has been built — and the recorder says what **is** happening, in the `status` message it pushes unprompted and again the moment a socket reconnects.

Whenever the two disagree, the page resends the missing message, with the same five fields. Repeats are safe: a second `start` for the same patient is `SESSION_ALREADY_ACTIVE`, and a second `prescription_built` is `GATE_ALREADY_ARMED`. Verified against a running recorder — section 22, `channel_a_test.py`.

What went wrong	Who notices	What happens, with nobody asked	Doctor sees
Recorder not running when the patient was opened	the page: the socket will not connect	**the page keeps trying every 2 s; on connect, sends start for the open visit**	a small "connecting" dot
Socket dropped at the moment of the click	the page: no reply within 5 s, or the socket closed	**the page resends start on reconnect**	nothing
Microphone busy	the recorder (`AGENT_NOT_READY` to the page)	**the recorder retries every 5 s for 15 min**; the page may also retry, harmlessly	nothing
Recorder crashed mid-consultation	the recorder, on restart	**records the same consultation again** if it was live within 10 min	a gap of about a minute
API 3a was lost	the page: status says `armed: false` but it built the prescription	**the page resends prescription_built**	nothing
Wrong patient opened first	nobody needs to	accepted; the server files the audio under the right patient (11.2)	nothing
Follow-up, no prescription	nobody needs to	accepted; the next patient's click is written down and the server gives each patient their own file (11.2)	nothing
Windows shuts down mid-consultation	the recorder	closes the recording cleanly; after a power cut, at most ~3 s is lost	nothing
Server says no	the recorder	**keeps recording; holds the audio on the PC**; asks again every 5 min	nothing
Configuration the page must fix	`ORIGIN_NOT_ALLOWED`, `DEVICE_NOT_ENROLLED`	**the page stops retrying** — a retry cannot fix configuration	a small banner that does not block work

Recording, but the microphone has gone silent	`level` at exactly 0 for 30 s; the recorder also alerts AIMS LAB	the page shows a small warning	"microphone?"

Why a late start still counts: start_time is always the original

When the page resends API 1 two minutes after the doctor opened the patient, it sends the **same five fields, including the original start_time** — not a new one. The recorder begins capturing from the moment the resend arrives, but the consultation is still identified by the moment it really began, so **API 2 attaches to it whenever API 2 arrives** — seconds, hours or days apart, in either order.

What is lost is only the audio between the click and the recovery — typically a few seconds. The consultation is not lost, and nobody had to do anything.

	Four rules that keep the page's reconciliation safe
1	**Never invent a prescription_built.** Resend only one the page actually sent, for the patient it was built for. The prescription decides whose recording it is
2	**Only reconcile the visit that is open now.** Once the doctor has moved to the next patient, the previous one is finished — do not resend its start
3	**Stop on configuration errors.** `ORIGIN_NOT_ALLOWED` and `DEVICE_NOT_ENROLLED` will fail every time
4	**Never block the doctor.** Every retry runs in the background; nothing modal, nothing that has to be dismissed

Known limitations on the AIMS LAB side

A microphone that dies mid-consultation — a headset unplugged, a driver crash — stops capture without reopening the device, and status still reports `is_recording: true`. The recorder alerts AIMS LAB after two silent pieces, and the `level` rule above is the page's signal.

A recorder that crashes mid-consultation leaves two recordings for the same five fields — the part before the crash and the part after. Both are kept; the clinical record attaches to one of them, the other stays unconfirmed, and the recorder's restart alert tells AIMS LAB they belong together.

PART D

The two server messages

API 2 and API 3 Channel B - every field, every limit, every reply

12 · API 2 — patient information

```
POST https://<aims-host>/api/v2/clinical/patient-information
X-CMED-Key: <key>

{
  "patient_id": "P0012345", "doctor_id": "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time": "2026-10-09T10:14:32+06:00", "date": "2026-10-09",

  "demographics": {
    "name": "Rahima Khatun", "sex": "female", "age_years": 34,
    "date_of_birth": "1992-03-11",
    "phone": "01700000000", "address": "Dholpur, Dhaka"
  },

  "paramedic": {
    "recorded_at": "2026-10-09T10:02:10+06:00",
    "weight_kg": 58.4, "height_cm": 157,
    "blood_pressure": "120/80", "pulse_bpm": 78,
    "temperature_c": 36.8, "spo2_percent": 98,
    "notes": "complains of headache"
  },

  "previous_visit": null,
  "female_details": {}, "male_details": {}
}
```

12.1 demographics — required

Field	Type	Accepted	If wrong	Lands in
name	text	up to 200. `full_name` also accepted	shortened	patients.full_name
sex	text	**exactly** `female` or `male`	**whole message quarantined**	patients.sex
age_years	number	0 to 130	stored empty	encounter_demographics
date_of_birth	text	YYYY-MM-DD	stored empty	patients.date_of_birth
phone	text	up to 40	shortened	patients.phone
address	text	up to 500	shortened	patients.address

demographics must be present on every visit, first or fiftieth. **sex is the one field that can fail the whole message** — send the word, not a code like M or F.

12.2 paramedic — optional, but send it when you have it

Field	Type	Accepted	Lands in
recorded_at	text	date and time **with offset**	paramedic_observations
weight_kg	number	0.5 to 499	weight_kg
height_cm	number	20 to 299	height_cm
blood_pressure	text	`"120/80"` — upper 30–300, lower 10–250	blood_pressure, also split into systolic and diastolic
pulse_bpm	whole number	10 to 300	pulse_bpm
temperature_c	number	25 to 45	temperature_c
spo2_percent	whole number	0 to 100	spo2_percent
notes	text	up to 2000	notes

A reading outside its range is stored empty — it never rejects the message. One mistyped temperature must not stop a prescription being saved.

Anything else inside `paramedic` is kept. If you measure something we have not listed — respiration rate, blood sugar, anything — send it. It is stored in an `other` field alongside the rest. **You do not need our permission and we do not need to change anything first.**

13 · API 3 Channel B — the prescription

```
POST https://<aims-host>/api/v2/clinical/prescription
X-CMED-Key: <key>

{
  "patient_id": "P0012345", "doctor_id": "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time": "2026-10-09T10:14:32+06:00",   <-- the ORIGINAL
  "date": "2026-10-09",

  "issued_at": "2026-10-09T10:26:55+06:00",
  "items": [
    { "drug": "Napa 500 mg", "dose": "1 tablet",
      "frequency": "1+1+1", "duration": "5 days",
      "instructions": "after food" },
    { "drug": "Omeprazole 20 mg", "dose": "1 capsule",
      "frequency": "1+0+0", "duration": "14 days" }
  ],
  "diagnoses":      ["Acute gastritis"],
  "investigations": ["CBC", "HbA1c"],
  "advice":         "avoid spicy food",
  "follow_up":      "2026-11-09",
  "notes":          "patient counselled about diet"
}
```

Field	Type	Required	Rule	Lands in
issued_at	text	**YES**	date and time **with offset**	prescriptions.issued_at
items	list	**YES**	one per medicine. **Send [] if none**	prescription_items
items[].drug	text	**YES**	up to 300. **Every item needs one**	drug
items[].dose	text	no	up to 200	dose
items[].frequency	text	no	up to 200, e.g. 1+1+1	frequency
items[].duration	text	no	up to 200, e.g. 5 days	duration
items[].instructions	text	no	up to 1000	instructions
diagnoses	list of text	**YES**	**Send [] if none**	diagnoses

investigations	list of text	**YES**	**Send [] if none**	investigations
advice	text	no	up to 2000	prescriptions.advice
follow_up	text	no	YYYY-MM-DD **if sent**	follow_up_date
notes	text	no	up to 2000	prescriptions.notes

An empty list and a missing field are not the same thing. `"diagnoses": []` means "the doctor recorded none"; leaving it out means "CMED did not send it", and that is reported as a problem. **Always send the list.** The order of `items` is preserved as the line number on the prescription.

14 · Corrections, repeats and retries

Situation	What to do	What happens
The doctor edits the prescription after sending	Send API 3 Channel B again with the **same five fields**	Stored as a new version. Both kept; the newest is current
The same prescription is rebuilt and sent again — a new `issued_at`, keys in another order	nothing special	`200 ALREADY_RECEIVED`: the same fingerprint (section 19.3) is the same prescription, so no new version
Your network failed and you are unsure	Send it again, unchanged	**Completely safe.** `200 ALREADY_RECEIVED` and one row, not two
You sent something wrong	Send the corrected version, same five fields	Same as an edit
You sent a field we do not recognise	nothing	Kept with the rest. **Unknown fields are never an error**

Every message is stored word for word, exactly as sent, before anything is read out of it. Nothing is ever lost because we failed to understand a field — if we later realise we should have been reading something, we can go back through the stored messages and get it.

15 · Every reply from the server

Code	Means	What CMED should do
202 ACCEPTED	Stored safely	**Remove it from the outbox.** For API 2, read `previous_visit.status` (19.2)
200 ALREADY_RECEIVED	Already held: this exact message, or (API 3) the same prescription by fingerprint	**Remove it and never send it again.** Retrying was safe
400 MALFORMED_JSON	Not valid JSON	Fix and resend. Do not retry unchanged
400 MISSING_FIELD	One of the five is missing or malformed	Fix and resend
400 INVALID_IDENTIFIER	An id has disallowed characters	Fix and resend
401 INVALID_KEY	Key missing or wrong	**Stop and contact AIMS LAB**
413 TOO_LARGE	Body over 1 MB	Check for a file or image in the JSON
422 SCHEMA_INVALID	**Stored in quarantine**, with the bad fields named	**Not lost.** Read the list, fix, resend; alert your team
429, 500, 502, 503, a timeout, no connection	Busy, a problem on our side, or no internet	**Keep it in the outbox and retry with growing delay.** Never drop it

No rate limit applies to CMED. Seven clinics at fourteen rooms produce roughly one message every two seconds at peak. Send as consultations happen — but from the outbox in section 16.

16 · Outages: the outbox on CMED's local server

Each Aalo branch runs CMED on its own local server, so the clinic keeps working through an internet or power cut and syncs to the cloud afterwards. **The recording side already works the same way**: the page talks to the recorder inside the laptop, the recorder keeps recording, holds the audio, and uploads when the internet returns. **But API 2 and API 3 Channel B leave from CMED's server, and while the internet is down they cannot reach us.** They must wait on CMED's side and go when it is back. That is the outbox.

	The outbox, as CMED's software team should build it
1	**A database table on the branch's local server** — not memory, not a browser. It must survive a power cut and a restart
2	**Written at the moment of the click**, in the same transaction that saves the visit or the prescription. The doctor never waits for the network
3	**Sent in order**: for one visit, API 2 before its API 3 Channel B; corrections after the original
4	**Retried with growing delay** while our server cannot be reached — for example 10 s, 30 s, 1 min, 5 min, then every 5 min — for as long as it takes. Hours or days are fine
5	**Each reply decides what happens next** — the table below
6	**Visible:** the number waiting and the age of the oldest, so a stuck outbox is noticed the same day

Reply	**The outbox does**
202 ACCEPTED	**remove** the message
200 ALREADY_RECEIVED	**remove it and never send it again** — we already hold it
400 MALFORMED_JSON / MISSING_FIELD / INVALID_IDENTIFIER	park it for a person to fix; **do not retry unchanged**; alert
422 SCHEMA_INVALID	stored in quarantine on our side, so not lost; park it, fix the shape, send the corrected one; alert
401 INVALID_KEY	**stop sending everything** and alert — the key is wrong or revoked; retrying cannot help
413 TOO_LARGE	park it; something too big is in the body; fix and resend
429, 5xx, timeout, no connection	**keep it and retry later**

API 2 arrives...	**What happens to the recording**
within a second or two	Its clinical record is attached at once
late — minutes, hours or days, after an outage	**Attached when it arrives.** The recording was recorded, uploaded and archived meanwhile; nothing has a deadline
never	**The recording is still kept and archived**, marked *unconfirmed*, and AIMS LAB is alerted. It simply has no clinical record — which is why a message must never be dropped

A message arriving a day late is completely fine. A message never sent is the only real failure. Nothing on our side deletes a recording because CMED's message was late or missing; the only thing that removes a recording is the patient refusing consent, on the doctor's Stop button.

17 · The reply that looks like success but is not

	Proves	Does NOT prove
`200 RECORDING_STARTED`	**The microphone is live** and audio is being written	that the consultation will be kept
`202 ACCEPTED`	**Your message is stored** and will not be lost	**that it matched a recording**

There is a third step, on our server: **we compare the five fields the recorder reported against the five in your message, and the clinical record is attached to the recording only if they are identical.**

The failure that matters, stated plainly

If your `start_time` differs from API 1's by even a second, or loses its `+06:00`, or the clinic code came from your own settings instead of the laptop:

- the recorder says **200 RECORDING_STARTED**
- our server says **202 ACCEPTED**
- **CMED sees nothing wrong at all**
- and the recording is kept **with no clinical record attached** — no demographics, no prescription

Two green replies mean both halves arrived. Whether they found each other is a separate question CMED cannot see the answer to. Section 22 is how to prove it.

PART E

Previous prescriptions

How history reaches us, and how to ask what we already hold

18 · History comes to us inside API 2

A returning patient's previous prescription reaches us as the `previous_visit` object inside API 2. **The field must always be present** — send `null` when there is nothing to send, because leaving it out cannot be told apart from "CMED forgot".

```
"previous_visit": {
  "doctor_id":   "DR0007",                    <-- that visit's OWN five
  "hospital_id": "AALO_DHOLPUR",                  fields, exactly as CMED
  "start_time":  "2026-06-02T09:40:05+06:00",     sent them at the time
  "date":        "2026-06-02",
  "prescription": {
    "items": [ { "drug": "Metformin 500 mg", "dose": "1 tablet",
                 "frequency": "1+0+1", "duration": "30 days" } ],
    "diagnoses": ["Type 2 diabetes mellitus"], "investigations": [],
    "advice": "review in one month", "follow_up": "2026-07-02"
  }
}
```

Field	Rule
date	**Required.** YYYY-MM-DD, the earlier visit's date, before this visit
doctor_id, hospital_id, start_time	**All three, or none.** With them we find the visit itself and never file it under today's doctor. Half a name is refused with 422
patient_id	optional; if sent, must be this patient
prescription	**Required**, the same shape as API 3 Channel B (section 13)

The date-only shape still works — `{"date", "prescription"}` — so nothing already built breaks, but it names no doctor, so we file it under `UNKNOWN` unless the same prescription is already held that day. Send the five fields.

19 · Ask first, and read what we did

19.1 What do you already hold for this patient?

```
GET https://<aims-host>/api/v2/clinical/patient/{patient_id}/visits?limit=50
X-CMED-Key: <key>

200 { "status": 200, "code": "OK", "patient_id": "P0012345",
      "visits": [
        { "patient_id": "P0012345", "doctor_id": "DR0007",
          "hospital_id": "AALO_DHOLPUR",
          "start_time": "2026-06-02T09:40:05+06:00", "date": "2026-06-02",
          "source": "live", "prescription_version": 2,
          "prescription_fingerprint": "f1e71094...ec4806",
          "received_at": "2026-06-02T03:58:11.204+00:00" } ],
      "next_cursor": null }
```

Parameter / reply	Meaning
`limit`	1 to 200, default 50. Newest first
`cursor`	When a page is full, `next_cursor` is the value to pass as `cursor` for the next page, unchanged
`before`	Optional, YYYY-MM-DD: only visits before that date
a patient we hold nothing for	`"visits": []`
errors	401 INVALID_KEY; 400 INVALID_IDENTIFIER; 400 MISSING_FIELD naming `limit` or `before`

Identifiers and fingerprints only — never a name, a medicine or a diagnosis, and no audio ever. Every request is logged against the patient.

	The rule for a returning patient
1	Ask for the visits
2	Compute the fingerprint (19.3) of the previous prescription as CMED holds it now
3	**Listed with the same fingerprint** → send `"previous_visit": null`. **Not listed, or a different fingerprint** → send `previous_visit`
4	**If the question cannot be asked** — our server down, a timeout — **send previous_visit anyway.** About 2 KB, and the reply says whether it was needed

19.2 What the API 2 reply says about previous_visit

```
202 { "status": 202, "code": "ACCEPTED", "record_id": "5121",
      "previous_visit": { "status": "ALREADY_STORED", "version": 2, ... } }
```

previous_visit.status	What we did	Meaning for CMED
ALREADY_STORED	**nothing** — held, same fingerprint	**That history is here.** No need to send it again
UPDATED	stored as that visit's **next version**; the newest is read	the corrected prescription is now held
STORED	stored that visit, under its real doctor and clinic	it was missing and is now held
NOT_SENT	—	previous_visit was null
UNIDENTIFIED	the date-only shape; `"stored"` says whether anything was added	send the five fields
PENDING	stored safely, still loading	nothing; it completes by itself

Today's patient information in the same API 2 is stored as usual whatever happens to previous_visit. A whole API 2 sent twice, byte for byte, is `200 ALREADY_RECEIVED`, with `ALREADY_STORED` for the history it had stored.

19.3 The prescription fingerprint — build it exactly like this

Step	Rule
1	Take these six keys and only these: `items` (in prescribed order; each entry exactly `drug, dose, frequency, duration, instructions`), `diagnoses`, `investigations` (lists of strings), `advice`, `follow_up` (YYYY-MM-DD), `notes`
2	Trim spaces at the ends of every text. An empty or missing text is `null`; a missing list is `[]`. An item with no drug is left out. A diagnosis or investigation sent as an object counts as its `text`, else its `name`
3	JSON with **keys sorted at every level**, no spaces (separators `,` and `:`), Bangla and other characters written as themselves, not `\u` escapes. UTF-8, then SHA-256, 64 lowercase hex

Worked example — reproduce this before using it. This prescription:

```
{ "issued_at": "2026-10-01T10:20:00+06:00",
  "items": [ { "drug": "Paracetamol", "dose": "500 mg",
               "frequency": "1+1+1", "duration": "5 days" } ],
  "diagnoses": ["Fever"], "investigations": ["CBC"],
  "advice": "Rest and fluids", "follow_up": "2026-10-08" }

becomes exactly this text (one line, shown wrapped):
{"advice":"Rest and fluids","diagnoses":["Fever"],"follow_up":"2026-10-08",
"investigations":["CBC"],"items":[{"dose":"500 mg","drug":"Paracetamol",
"duration":"5 days","frequency":"1+1+1","instructions":null}],"notes":null}

fingerprint:
f1e7109404fd5d4010c358f0b3afdec79699955299a089eaf2c830fb60ec4806
```

`issued_at`, key order, spacing and any field not listed make no difference. A change to any medicine, its order, a diagnosis, the advice, the follow-up or the notes is a different fingerprint. Our test suite pins the same value.

PART F

The data model

Where every field lands, and what the two databases hold

20 · Where every field ends up

So CMED can see that nothing is discarded. All of this lives in a database called `aims_clinical`, separate from the one that tracks recordings — which deliberately holds no patient names at all.

You send	We store it in	Holding
the whole message, untouched	`intake_records`	**exactly what you sent**, once per message, before anything is parsed
demographics	`patients`, `encounter_demographics`	name, sex, date of birth, phone, address, and the age at this visit
paramedic	`paramedic_observations`	each reading in its own column, blood pressure also split into systolic and diastolic, anything unrecognised in `other`
the visit itself	`encounters`	one row per consultation, linked to its recording
previous_visit	`encounters`, `prescriptions`	**that earlier visit's own row** — the one held, or a new one under its real doctor — and a new prescription version only if it changed
items	`prescription_items`	**one row per medicine**, in the order you sent them
the prescription	`prescriptions`	with a version number, so corrections never overwrite
diagnoses	`diagnoses`	one row each
investigations	`investigations`	one row each
female_details / male_details	their own tables	stored as sent

21 · The two databases, and why they are separate

	`aims_clinical`	`aims_recordings`
Holds	**Everything CMED sends** — patients, demographics, paramedic readings, prescriptions, previous visits	The audio tracking record — sessions, segments, hashes, archive state
Patient names?	**yes**	**no — by design, none at all**
Size	about 76 KB per consultation; roughly 10 GB for the study	small — metadata only
Who writes it	the two endpoints in this document	the recorders

The split is deliberate. A recording can be tracked, verified, archived and audited end to end without ever touching a patient name, which means the operational half of the system carries no identifying data.

A copy of every consultation's clinical record is **also written as a JSON file beside the audio**, with the same name. So each consultation is one folder containing the audio and its record together, and the database is an index into them — if the database were lost entirely, the dataset would still be readable.

PART G

Testing and handover

What to run, what to send us, and the page to keep on the desk

22 · Testing, in the order that makes sense

	What	Needs	How
1	**The browser half — all of it**	**a PC with the recorder installed. No internet**	**Available today.** `python tools/channel_a_test.py --origin https://<your page>` — 22 checks, including both recovery cases in section 11A
2	Check a server message against the rules	nothing at all	`python tools/channel_b_test.py --validate my.json --kind patient_information`
3	**The server half against a live host**	**a staging hostname from us**	`channel_b_test.py --server https://<host> --key <key>`
4	Watch every message and reply side by side	the staging site	open `/protocol-test`
5	**A real consultation, all four messages**	a clinic PC	and then step 6 — do not skip it
6	**AIMS LAB confirms it matched and archived**	**ask us**	**the only check that proves the integration**

Step 1 is unblocked today and is the half with the awkward detail — the gate, the session id, the provisional reply, reading the clinic code. Start there while we publish a staging host for step 3. **Please run step 3 and send us the output before the first clinic opens**; it takes under a minute and has caught every integration problem we have seen.

Step 6 must be asked for explicitly. Steps 1 to 5 can all pass while the five fields silently fail to match. We can see the answer on the dashboard in seconds — it is the only green light that means anything.

22A · Five commands that test the server half, before you write code

These need no application and no page. Run them as soon as you have the staging hostname and the key.

Is it up?

```
curl -s https://<aims-host>/health | jq .
# expect: "status": "healthy"
#   and   "system" STARTS WITH "AIMScribe v3"
#         (today it reads "AIMScribe v3 - AIMS LAB server")
```

Is my key accepted? Send a patient

```
curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST https://<aims-host>/api/v2/clinical/patient-information \
  -H "X-CMED-Key: $CMED_KEY" \
  -H "Content-Type: application/json" \
  -d '{
        "patient_id":  "TEST0001",
        "doctor_id":   "DR0042",
        "hospital_id": "AALO_DHOLPUR",
        "start_time":  "2026-10-09T10:14:32+06:00",
        "date":        "2026-10-09",
        "demographics": { "name": "Test Patient", "sex": "female",
                          "age_years": 34 },
        "previous_visit": null
      }'
```

```
# expect: 202
```

Send that exact command twice. The first gets **202**, the second **200** — the second is `ALREADY_RECEIVED`. That proves retries are safe, which is the most useful thing to know before writing your queue.

Does validation work? Break it deliberately

```
# previous_visit omitted on purpose, and sex is a code rather than a word
curl -s -X POST https://<aims-host>/api/v2/clinical/patient-information \
  -H "X-CMED-Key: $CMED_KEY" -H "Content-Type: application/json" \
  -d '{ "patient_id":"TEST0002", "doctor_id":"DR0042",
        "hospital_id":"AALO_DHOLPUR",
        "start_time":"2026-10-09T10:20:00+06:00", "date":"2026-10-09",
        "demographics": { "name":"Test", "sex":"F" } }' | jq .
# expect: 422 SCHEMA_INVALID, naming demographics.sex and previous_visit
```

The reply names the bad fields. Build against that — it is faster than reading this document, and the record is quarantined rather than lost, so nothing is wasted by experimenting.

What do you hold for this patient?

```
curl -s -H "X-CMED-Key: $CMED_KEY" \
  https://<aims-host>/api/v2/clinical/patient/TEST0001/visits | jq .
# expect: 200, "code": "OK", the visits held, each with a fingerprint
# without the key: 401 INVALID_KEY
```

Send the prescription

```
curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST https://<aims-host>/api/v2/clinical/prescription \
  -H "X-CMED-Key: $CMED_KEY" -H "Content-Type: application/json" \
  -d '{
        "patient_id":  "TEST0001",
        "doctor_id":   "DR0042",
        "hospital_id": "AALO_DHOLPUR",
        "start_time":  "2026-10-09T10:14:32+06:00",   <-- SAME as API 2
        "date":        "2026-10-09",
        "issued_at":   "2026-10-09T10:26:55+06:00",
        "items": [ { "drug": "Napa 500 mg", "dose": "1 tablet",
                     "frequency": "1+1+1", "duration": "5 days" } ],
        "diagnoses":      ["Acute gastritis"],
        "investigations": []
      }'
# expect: 202
```

The one thing these commands cannot prove

All of them can pass while the integration is still broken, because **a 202 means "stored", not "matched to a recording"**. If `start_time` differs by a second between API 1 and API 2, or loses its offset, every reply above is still green and **the recording ends up with no clinical record attached**.

So the real test needs a recorder running and our confirmation — steps 5 and 6 of section 22. Please do not treat green curls as integration complete.

23 · A complete worked example, with recovery built in

The browser half of the integration as it should actually be written: the five fields built once, and **a reconcile step that resends whatever the recorder is missing** — so a recorder that was not running or a dropped socket heals itself without the doctor knowing, on top of what the recorder recovers by itself. Section 11A is the reasoning; this is the code.

```
// ---- what SHOULD be happening: the page's own record --------------------
let visit = null;          // the open consultation: the five fields, built once
let built = false;         // has the doctor built ITS prescription?
let CLINIC = null;         // read from the recorder, never hardcoded
let socket = null, sessionId = null, stopRetrying = false, retryMs = 2000;

const CONFIG_ERRORS = ["DEVICE_NOT_ENROLLED", "ORIGIN_NOT_ALLOWED"];

function send(command, extra) {
  if (!socket || socket.readyState !== WebSocket.OPEN) return false;
  socket.send(JSON.stringify({ command, request_id: command + "-" + Date.now(),
                               ...extra }));
  return true;
}

// ---- the heart of it: compare SHOULD with IS, resend whatever is missing --
function reconcile(status) {
  if (!visit || stopRetrying) return;
  if (!status.is_recording) {                    // nothing recording at all
    send("start", { trigger: visit });           // SAME five fields - safe
    return;                                      // SESSION_ALREADY_ACTIVE if dup
  }
  // Recording that started with another patient is fine: our start was
  // accepted into it (continued: true), and the server sorts out whose is whose.
  sessionId = status.session_id;
  if (built && !status.armed) {                  // our 3a was lost
    send("prescription_built", { patient_id: visit.patient_id,
                                 session_id: sessionId });   // safe if dup
  }
}

function connect() {
  socket = new WebSocket("ws://127.0.0.1:5050/ws");

  socket.onopen = () => { retryMs = 2000; showConnected(true);
                          send("doctors"); };     // clinic code first

  socket.onmessage = (e) => {
    const m = JSON.parse(e.data);

    // Status reaches us two ways: pushed unprompted (event "status"), and as
    // the reply to our own poll (command "status", the fields under data).
    const status = m.event === "status" ? m
                 : m.command === "status" ? m.data : null;
    if (status) {
      showRecordingState(status);                // incl. a level meter
      reconcile(status);                         // <-- self-healing
      return;
    }
    if (m.command === "doctors") {
      CLINIC = m.data.hospital_id;
      if (!CLINIC) { stopRetrying = true; return banner("PC not registered"); }
      showClinicName(CLINIC);
      fillDoctorSelector(m.data.doctors);        // [] is normal
      return;
    }
    if (m.command === "start") {
      if (m.code === "RECORDING_STARTED" || m.code === "RECORDING_PROVISIONAL" ||
          m.code === "SESSION_ALREADY_ACTIVE") {  // all three mean: recording,
                                                  // keep the session it gave us
        sessionId = m.data && m.data.session_id || sessionId;
      } else if (CONFIG_ERRORS.includes(m.code)) {
        stopRetrying = true; banner(m.code);     // a retry cannot fix this
      } else if (m.code === "AGENT_NOT_READY") {  // the recorder retries too;
        setTimeout(() => send("start", { trigger: visit }), retryMs);  // harmless
```

```
        retryMs = Math.min(retryMs * 2, 30000);   // 2, 4, 8, 16, 30, 30...
      }
    }
  };

  socket.onclose = () => { showConnected(false);
                           setTimeout(connect, 2000); };  // quietly, forever
}

// ---- the doctor opens a patient ----------------------------------------
async function openPatient(patient, doctor) {
  visit = {                                       // built ONCE, here
    patient_id:  patient.id,
    doctor_id:   doctor.id,
    hospital_id: CLINIC,
    start_time:  new Date().toISOString(),        // the ORIGINAL, forever
    date:        localDateString()
  };
  built = false; stopRetrying = false; retryMs = 2000;
  send("start", { trigger: visit });              // if the socket is down,
                                                  // reconcile() sends it later
  outbox.add("API2", visit, demographics, paramedic,  // section 16: a table on
             await previousVisitIfNeeded(patient));    // the branch server (19.1)
}

// ---- the doctor builds the prescription - this does NOT stop recording -----
// visit is the patient the prescription is for; if a wrong profile was
// opened first, the server files the audio under this patient.
function prescriptionBuilt(prescription) {
  built = true;                                   // remembered, so it can be
  send("prescription_built", {                    // resent if it gets lost
    patient_id: visit.patient_id, session_id: sessionId });
  outbox.add("API3B", visit, prescription);       // same visit object
}

// A safety net for the one path with no status event: an idle recorder
// whose socket is open but which never received the start.
setInterval(() => send("status"), 10000);

connect();
```

The single most important function is `reconcile()`. It runs on every status message — including the one the recorder sends the instant a socket reconnects — and it is the reason the page's half of the doctor never having to re-open a patient. Note that it resends the **same** `visit` object, so start_time is still the original and the late recording still matches API 2.

24 · What each side must hand over

AIMS LAB gives CMED	When
The staging and production hostnames	with the key
The `X-CMED-Key` — shown once	on request, out of band
The seven clinic codes	with the key
`channel_b_test.py`	with the key
Confirmation a test recording matched and archived	**on request, step 6**

CMED gives AIMS LAB — both blocking	Why it blocks
1. Every exact web address your page is served from, production, staging and preview	**Without it the page cannot open the local connection at all.** Nothing works. `www` and non-`www` differ, as do `http` and `https` and any non-default port
2. Your own clinic codes, so we can map them to ours	**Without the mapping a visit cannot be tied to the clinic it happened in. The record is stored and you receive a normal 202, but it is never attached to its recording.** Silent from CMED's side; we are alerted
A named technical contact	So a question either way does not wait a week
A staging environment we can send to	So we can test our side against yours

25 · One page for the CMED developer

Question	Answer
How many messages do I build?	**Four.** Two from the browser, two from your server
What ties them together?	**The five fields, identical in all four**
Biggest trap	**start_time in API 3 is the original**, not when the prescription was built
Where do I connect, browser side?	`ws://127.0.0.1:5050/ws` — the same PC, every time
Do I need a key there?	**No.** We register your page's web address instead
Where do I POST, server side?	`/api/v2/clinical/patient-information` and `/api/v2/clinical/prescription`
How do I authenticate there?	`X-CMED-Key`. Server to server only, never in a browser
What do I keep from API 1?	`session_id` — API 3 Channel A needs it
Where does hospital_id come from?	**The `doctors` command.** Never hardcoded
Is RECORDING_PROVISIONAL bad?	**No** — it means recording. Our server never stops it afterwards
Another patient opened mid-recording?	**Accepted** (RECORDING_STARTED, continued). Send their API 2 as usual (11.2)
Does prescription_built stop it?	**No.** It arms the gate; the next patient closes the consultation
Can I send stop or pause?	**No.** Those belong to the doctor
Required in API 2?	`demographics`, and `previous_visit` present (null is fine)
Required in API 3 Channel B?	`issued_at`, `items`, `diagnoses`, `investigations` — lists may be empty
Out-of-range reading?	Stored empty. It never rejects the message
Extra fields?	Kept. Never an error
Is retrying safe?	**Yes.** One row, and `ALREADY_RECEIVED` — then remove it from the outbox

Internet down at the branch?	**Keep both server messages in the outbox** and send when it is back. Late is fine (16)
422?	**Nothing lost.** Quarantined, with the bad fields named
Do you already hold the previous visit?	**Ask:** `GET .../visits`, and compare fingerprints (19)
Does a 202 mean it worked?	**No — stored, not matched.** Section 17
What must I send AIMS LAB?	**Your page's exact addresses, and your clinic codes.** Both blocking
Can I start today?	**Yes — the whole browser half.** Section 22

Prepared by AIMS LAB, United International University. **Document 1 of 2**; the companion is "Server Deployment", for the team helping us deploy. Every rule here is taken from the code that handles these messages. If anything is unclear, ask before building — it is far cheaper than discovering it in a clinic.